

Проект “Chess”

Описание проекта

Реализовать игру в шахматы на 2-х игроков. Проект должен содержать стартовое меню и окно партии. При закрытии окна партии прогресс должен сохраняться на компьютере в файловую систему.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий положение и методы для движения (и, если нужно, атаки) фигуры.

Абстрактный класс фигуры, реализующий интерфейс и добавляющий необходимые свойства и методы.

Классы-наследники для каждого вида фигур с движением на свободные клетки согласно их виду.

Стартовое меню с 2-мя кнопками: “Новая игра” и “Продолжить игру”. Новая игра начинается с базовой шахматной расстановки белыми фигурами, а продолжение игры - с сохраненной партии, начиная с игрока, чей ход был до выхода.

В новом окне должно открываться игровое поле с фигурами. Управление мышью: выбор фигуры, перемещение фигуры. После этого происходит переход хода другому игроку.

Создать класс для сериализации состояния игры в формате JSON. Сохранение игры осуществлять при закрытии окна игры.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей игры. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “Chess”.

Для класса, отвечающего за логику партии добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации окончания игры при условии мата: король противника не может совершить ход и не может быть закрыт другой фигурой от угрожающей ему фигуры противника.

Сообщать пользователю о шахе (с указанием цвета фигур).

Сообщать пользователю о мате (с указанием цвета фигур).

Добавить подсветку клеток, доступных для перемещения выбранной фигуры.

Добавить отмену хода: если поставить фигуру на то место, где она стояла, то можно походить другой фигурой.

В стартовом меню добавить поле для выбора папки для сохранения игры.

Продвинутая часть (2 балла):

Если выбранный файл сохранения не соответствует возможному варианту для игры (выбран какой-то сторонний файл), кнопка “Продолжить игру” должна быть неактивной. Также пользователю должно сообщать о том, что выбран файл некорректного формата.

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за фигуры и работу с ними, ход игры и т. п. поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Создать ещё одну часть этого класса partial для реализации окончания игры при условии пата: противник совершает 6 взаимно обратных действия (перемещение с одной клетки на другую и обратно).

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять прогресс активной партии. При смене формата копировать все данные из “текущего формата” в файл “нового формата”.

Проект “MusicalTails”

Описание проекта

Реализовать игру Musical Tiles или любой из его аналогов. Проект должен содержать стартовое меню и окно игры. При закрытии окна игры попытка считается законченной, счет добавляется в таблицу лучших 10 игр, если превышает какой-либо из таблицы результатов игр.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий действия, возможные при нажатии на музыкальные кнопки.

Абстрактный класс, реализующий интерфейс и добавляющий необходимые свойства и методы.

Классы-наследники для каждого вида кнопок: короткая, требующая нажатия и длинная, требующая удержания.

Для генерации клавиш должен быть отдельный класс.

Стартовое меню с кнопкой: “Начать игру”. Таблица лучших 10 игр (при запуске приложения подтягивать данные таблицы из файла на компьютере). При первом запуске создать пустой файл и пустую таблицу.

Очки начисляются за каждое успешное нажатие кнопки.

Новая игра начинается в новом окне, где имеется несколько дорожек и сверху вниз падают кнопки (клавиши, кружки и т.п.), на которые нужно нажимать.

Ограничить область нажатия в нижней части экрана.

При закрытии окна игры попытка считается законченной и счет добавляется в таблицу, а также с файл в формате JSON, если он входит в топ-10 результатов.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей игры. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “MusicalTails”.

Для класса, отвечающего за логику обработки клавиш добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации окончания игры при условии: пропущена хотя бы одна кнопка (оказалась ниже кликабельной области).

Сообщать пользователю о проигрыше (с указанием счета).

Добавление “усложнения”: чем выше счет, тем выше скорость генерации и движения кнопок.

В главном меню добавить выбор сложности: выпадающий список или слайдер со значениями от 2 до 8 - количество полос для кнопок. Счет за нажатие кнопки должен умножаться на количество полос.

Продвинутая часть (2 балла):

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить новый вид (класс) кнопки, который требует двойного нажатия на него. Давать больше очков за такие кнопки. Но они должны быть более редкими (вероятность появления растет с увеличением счета) Отображать такие кнопки другим цветом.

Добавить новый вид (класс) кнопки-ловушки. На такие кнопки не следует нажимать.

Добавить 3й partial класс для обработки клавиш: при нажатии на кнопку-ловушку, заканчивать игру. А при перемещении вниз области игры начислять очки. Скорректировать логику завершения игры во 2м partial классе.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять результат лучших 10 игр. При смене формата копировать все данные игр из "текущего формата" в файл "нового формата".

Проект “CollegeProgram”

Описание проекта

Реализовать электронную ведомость по группам ВУЗа. Проект должен содержать главное меню, представляющую собой набор фильтров для поиска и добавления или удаления групп из различных институтов (факультетов). Фильтры также должны влиять друг на друга.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий группу студентов с действиями зачисления в группу, перевода в другую группу и отчисления из группы.

Класс для предмета: название, номер семестра в котором он читается и количество часов.

Класс для образовательной программы, содержащей название и массив предметов.

Классы группы (реализующей интерфейс), института (абстрактный с наследниками, имеющие массив различных программ обучения).

При первом запуске приложения создать все институты и заполнить их образовательными программами автоматически.

В главном меню сделать выпадающий список для институтов. Сделать таблицу групп, содержащихся в данном институте с возможностью добавления новой группы в институт с указанием названия и образовательной программы, к которой она принадлежит.

При любом изменении состава института сохранять данные в файл в формате JSON. При выборе института читать информацию из соответствующего файла.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “CollegeProgram”.

Для класса, отвечающего за группу добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации перевода группы на новую образовательную программу (в рамках одного института).

В главном меню сделать выпадающий список для образовательных программ. Добавить переключатель, по какому параметру формировать таблицу групп: институту или образовательной программе. При фильтрации по образовательной программе при добавлении в таблицу новой группы образовательная программа должна подставляться автоматически в соответствии с выбранной в фильтре.

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить 3й partial класс для группы: перенос её в другой институт на новую образовательную программу.

Добавить в главном меню поле для названия группы и кнопку поиска. В таблицу должны быть выведены все группы, которые в названии содержат содержимое заполненного поля.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять информацию из таблицы групп. При смене формата копировать данные из всех файлов "текущего формата" в файлы "нового формата".

Проект “TrainingSection”

Описание проекта

Реализовать приложение для сбора статистики эффективности работы тренеров. Проект должен содержать главное меню, представляющую собой набор фильтров для поиска, добавления и удаления групп тренеров и участников групп. Фильтры также должны влиять друг на друга.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий человека.

Класс, реализующий интерфейс, классы-наследники для спортсмена и тренера.

Класс для группы, в которой есть 1 тренер и массив спортсменов.

При первом запуске приложения создать не менее 3-х тренеров и не менее 5 групп автоматически. Обязательно должны быть несколько групп, у которых общий тренер.

В главном меню сделать выпадающий список для групп. Сделать таблицу для участников группы. Содержимое таблицы должно обновляться при изменении группы. В группу можно через таблицу добавлять участников

При любом изменении состава группы сохранять данные в файл в формате JSON. При выборе группы читать информацию из соответствующего файла.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “TrainingSection”.

Для класса, отвечающего за группу добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации метода сортировки группы по фио и методов фильтрации (выбора из всех участников части) по полу и по возрасту.

В главном меню сделать выпадающий список тренеров. Создать таблицу, которая показывает, какие группы ведет выбранный тренер, количество участников и % соотношение мужчин и женщин в группе, а также средний и медианные возраста. Менять содержимое таблицы нельзя.

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить 3й partial класс для группы: свойство для количество занятий в группе и метод для оценки тренера после прохождения курса тренировок. При выставлении оценки указывается количество занятий с тренером и оценка его работы. Из этих оценок формировать рейтинг у тренеров.

В главном меню сделать таблицу тренеров с указанием их рейтинга. Создать выпадающий список всех спортсменов. Выбрав спортсмена и группу должна становиться активной кнопка “Оставить фидбек”. При нажатии на эту кнопку должен вызываться метод для оценки тренера. Больше этот спортсмен не может оставить оценку на данную группу.

Создать интерфейс **IReportable**, содержащий метод **GenerateReport()**. Реализовать этот интерфейс в классах спортсмена, тренера и группы. При вызове выводить необходимую информацию (например: фио, рейтинг, количество участников).

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять отчет по всем тренерам из таблицы. При смене формата копировать данные из всех файлов “текущего формата” в файлы “нового формата”.

Проект “LotteryArchive”

Описание проекта

Реализовать приложение для архивации данных о победителях различных лотерей. Проект должен содержать главное меню, представляющую собой набор фильтров для поиска, диаграммы или таблицы, или графика для визуализации данных.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий человека.

Класс, реализующий интерфейс, и его класс-наследник для участника лотереи.

Класс для билета и его класс-наследник для выигрышного билета.

Класс для лотереи, определяющий количество билетов (сколько цифр может быть в названии), призовой фонд.

В главном меню сделать настройку для создания лотереи: название, количество билетов, количество участников, призовой фонд. Автоматически создавать участников лотереи, которые покупают случайное количество билетов. Один билет может быть только у одного участника.

После этого определить победителя данной лотереи и его выигрыш (если он зависит от количества участников).

После этого сохранять данные о данной лотереи в файл в формате JSON.

Добавить кнопку “Посмотреть статистику”. Она должна выводить информацию из файлов JSON в виде таблицы.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “LotteryArchive”.

Для класса, отвечающего за лотерею добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации методов продажи билетов участникам (то есть один участник сможет докупать билеты). Для класса участника лотереи добавить модификатор partial. Создать ещё одну часть этого класса со свойствами для “жадности” и “баланса”, чем больше “жадность”, тем больше он готов потратить денег на билеты. Для класса билета добавить модификатор partial. Создать ещё одну часть этого класса с полем для “цены” билета. Изменить логику покупки билетов с использованием новых частей классов.

В главном меню сделать таблицу людей. В неё можно добавлять или менять данные людей: ФИО, жадность, баланс. В новых лотереях могут участвовать только люди из таблицы. При участии их балансы должны меняться.

Продвинутая часть (2 балла):

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить 3й partial класс для лотереи: выигрышный билет обязательно должен формироваться из списка распроданных билетов, если было распродано больше 25% билетов. Иначе отменять лотерею. И возвращать всем участникам 90% потраченных денег.

В главном меню сделать чекбокс для списка людей. Для выбранных людей создавать диаграммы или графики потраченных на билеты и выигранных денег.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять информацию для новой лотереи. При смене формата копировать данные из всех файлов "текущего формата" в файлы "нового формата".

Проект “DoodleJump”

Описание проекта

Реализовать игру Doodle Jump. Проект должен содержать стартовое меню и окно игры. При закрытии окна игры прогресс должен сохраняться на компьютере в файловую систему.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Класс для персонажа игры с возможностью перемещения по горизонтали.

Создать интерфейс, описывающий площадку, на которой можно прыгать.

Класс площадки, на которой может прыгать персонаж, реализующий интерфейс и добавляющий необходимые свойства и методы.

Классы-наследники для обычной площадки, разрушаемой площадки (после 1 отталкивания). У разных наследников площадки должны быть разных цветов.

Класс для реализации карты и логики игры: генерация площадок, их движение (имитация движения персонажа по вертикали), остановка игры, если персонаж касается нижней границы экрана.

Стартовое меню с 2-мя кнопками: “Новая игра” и “Продолжить игру”. Новая игра начинается с начальной площадки. Остальные площадки должны генерироваться случайно. Продолжение начинается с сохраненной позиции площадок и персонажа.

Управление персонажем осуществлять стрелками влево-вправо или буквами “A” “D” на клавиатуре.

Класс для сериализации состояния игры в формате JSON: положение персонажа и площадок, а также счет. Счет считать при перемещении персонажа вверх как максимально достигнутую высоту относительно стартовой площадки.

Сохранение игры при закрытии окна игры.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей игры. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “DoodleJump”.

Для класса, отвечающего за логику игры добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации перемещения между краями (закольцованность): персонаж, пересекая левую границу поля, должен появляться справа. Ограничить диапазон генерации площадок так, чтобы они не выходили за края.

Добавить класс-наследник для площадки, позволяющий прыгнуть сильно выше (на 10+ площадок).

При прохождении через площадку снизу-вверх персонаж должен пролетать её не задевая, только оказавшись под персонажем, площадка должна становиться активной для взаимодействия.

В стартовом меню добавить поле для выбора папки для сохранения игры.

Продвинутая часть (2 балла):

Если выбранный файл сохранения не соответствует возможному варианту для игры (выбран какой-то сторонний файл), кнопка “Продолжить игру” должна быть неактивной. Также пользователю должно сообщать о том, что выбран файл некорректного формата.

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за фигуры и работу с ними, ход игры и т. п. поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять прогресс активной партии.

Добавить класс-наследник для площадки, от которой нельзя оттолкнуться, она сразу ломается при взаимодействии, персонаж продолжает падение вниз.

Добавить “физику” в движение персонажа - постепенное замедление скорости при прыжке вверх и увеличение скорости движения вниз со временем.

Проект “RestaurantMenu”

Описание проекта

Реализовать приложение для составления меню ресторанов, кафе, кофеен. Проект должен содержать главное меню, представляющую собой действия для выбора кафе и окно для открывающегося меню.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Класс блюда с указанием названия и цены, от него классы-наследники для горячих блюд, напитков, десертов и т.п.

Создать интерфейс, описывающий меню, содержащий массив блюд.

Класс, реализующий интерфейс с методами для изменения содержания меню.

Класс для заведения и его классы-наследники для ресторана, кафе и кофейни.

Программно создать несколько заведений разных видов и добавить им различные меню.

В главном меню сделать выбор заведений из списка. При выборе заведения становится активной кнопка “Показать меню” для открытия меню. При нажатии должно отображаться меню (например таблицей), в котором прописывается перечень блюд. Данные о заведении читать из файла в формате JSON (создать файлы при создании заведений).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “RestaurantMenu”.

Для класса, отвечающего за меню добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации методов добавления или удаления позиций из меню.

Создать интерфейс со свойством для сезонного меню и методами для добавления и удаления сезонного меню.

Для класса заведения добавить модификатор partial и реализовать вышеуказанный интерфейс.

В окне меню добавить возможность добавлять или удалять позиции из меню. Добавить кнопку сохранения, при нажатии на которую, меню *для данного заведения* должно сохраняться в файл.

В главном меню сделать второй выпадающий список для выбора вида меню после выбора заведения: обычное или сезонное.

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить 3й partial класс для класса заведения: добавить в него метод **Select**, который получает на вход вид позиций (Type или string) и возвращает из меню только позиции подходящего вида.

В главном меню сделать выпадающий список для выбора заведений: ресторанов, кафе и кофеен. В выпадающем списке должны быть заведения только выбранного типа. Если ничего не выбрано, то должны быть доступны все.

При нажатии на кнопку “Показать меню” меню должно открываться в новом окне. В этом окне добавить фильтр для выбора блюд (например разделение можно сделать: завтраки, горячее, закуски, напитки).

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять меню. При смене формата копировать данные из всех файлов “текущего формата” в файлы “нового формата”.

Проект “ScientistPlatform”

Описание проекта

Реализовать приложение для хранения и анализа научных статей. Проект должен содержать главное меню для выбора параметров для поиска статей и сам поиск.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Класс для автора с уникальным полем **ORCID**.

Создать интерфейс для статьи со свойством **Keywords** для массива ключевых слов (и прочими необходимыми полями и свойствами) и класс, реализующий данный интерфейс с уникальным свойством **ISSN**. А также наследники:

ResearchArticle: представляющий научные исследовательские статьи.

ReviewArticle: представляющий обзорные научные статьи.

CaseStudy: представляющий статьи о конкретных случаях исследований.

Программно создать несколько авторов и статей (в статье должно быть от 1 до 10 авторов).

В главном меню сделать выбор критерия для поиска (публичные свойства статьи) и поле поиска. При заполнении критерия и значения становится активной кнопка “Найти статьи”. Найденные статьи отображать таблицей. Данные о статьях читать из файлов в формате JSON (создать файлы при первом запуске).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “ScientistPlatform”.

Создать класс для издательства со свойством рейтинга и списка тем.

Для класса, отвечающего за статью добавить модификатор partial. Создать ещё одну часть этого класса partial для добавления издательства. Добавить можно только, если статья подходит к какой-либо теме издательства.

В главном меню добавить список издательств. Добавить поле для указания **ISSN** статьи и кнопку для отправки данной статьи в издательство.

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать интерфейс **ICitation** со свойством оформления ссылки на статью.

Добавить 3й partial класс для класса статьи. В нем реализовать интерфейс **ICitation**. Для разных издательств должен быть различный формат ссылки при одних и тех же прочих параметрах.

В главном меню сделать кнопку для создания ссылки. Открывается новое окно. Там выбирается статья из списка и издательства. При нажатии на кнопку “Создать ссылку на статью” должна формироваться строка с ссылкой в поле. Добавить кнопку для сохранения ссылки в txt файл.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять данные статей. При смене формата копировать данные из всех файлов “текущего формата” в файлы “нового формата”.

Проект “FoodQualityAnalyzer”

Описание проекта

Реализовать приложение для анализа качества продуктов. Проект должен содержать главное меню для выбора продуктов и окно с графическим отображением характеристик продуктов.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **FoodProduct** со свойствами для названия, количества дней до истечения срока, максимальный срок годности любыми необходимыми дополнительными полями. Определите в этом классе абстрактный метод **GetQuality()**, который будет возвращать число, описывающее качество продукта.

Создать классы, наследующиеся от **FoodProduct**, представляющие различные виды продуктов питания, такие как **Vegetable**, **Fruit**, **Meat**, **Backery**. Каждому из наследников добавьте по 2 уникальных параметра.

Создать интерфейс **ISpreadable** с массивом продуктов и методами для добавления одного или нескольких продуктов. Класс **FoodQualityAnalyzer**, его реализующий.

Программно создать несколько (10-20 различных) продуктов.

В главном меню сделать выбор продуктов из списка (чекбокс). При заполнении хотя бы одного продукта становится активной кнопка “Показать качество”. В новом окне должна быть диаграмма (круговая или столбчатая). Данные о продуктах читать из файлов в формате JSON (создать файлы при первом запуске). Данные полученной диаграммы создавать в файлы с отчетами в стиле: “Отчет_№X_от_data” (или любом похожем).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “FoodQualityAnalyzer”.

Создать интерфейс **IShrinkable** с методами для удаления конкретного продукта или продукта под определенным номером или последнего продукта из списка с соответствующими параметрами: (**FoodProduct**), (**int**) и (**void**).

Для класса **FoodQualityAnalyzer** модификатор partial. Создать ещё одну часть этого класса partial, реализующий интерфейс **IShrinkable**.

Сохранять параметры в списке продуктов после закрытия окна с отчетом. Если снимать галочки, то убирать из списка продуктов продукты (да, изменить логику, если раньше добавление было в момент нажатия на кнопку “Показать качество”).

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать интерфейс **IStatistic** с методами, которые будут рассчитывать наивысшее, наименьшее, среднее и медианное качество продуктов в списке.

Добавить 3й partial класс для класса статьи. В нем реализовать интерфейс **IStatistic**.

В главном меню сделать кнопку для создания продукта. Открывается новое окно. В нем должен быть выпадающий список для типов продуктов, поле для названия, сроком годности и прочими. При нажатии на кнопку "Создать продукт" должен формироваться новый объект и добавляться в чекбокс на главной странице. Также должен создаваться новый файл в формате json и xml.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять отчет. При смене формата копировать данные из всех отчетов "текущего формата" в файлы "нового формата".

Проект “ComputerGames”

Описание проекта

Реализовать приложение для поиска компьютерных игр. Проект должен содержать главное меню для выбора продуктов и окно с изображением (ссылки на источники можно указать в Readme) и описанием игры.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **Game** со свойствами для названия, жанра, возрастного ограничения, даты выхода и прочими необходимыми дополнительными свойствами.

Создать интерфейс **IPlatform**.

Создать интерфейсы **IConsoleable**, **IComputerable** и **IMobile**, наследующиеся от **IPlatform** и определяющие методы для игр на различных платформах.

Создать три класса, наследующихся от **Game**, представляющих различные типы компьютерных игр: **SingleGame**, **MultiplayerGame** и **OnlineGame**.

Класс **SingleGame** должен реализовывать интерфейсы **IComputerable** и **IConsoleable**.

Класс **MultiplayerGame** должен реализовывать все три интерфейса: **IConsoleable**, **IComputerable** и **IMobile**.

Класс **OnlineGame** должен реализовывать интерфейсы **IComputerable** и **IMobile**.

Создать partial класс **GameCatalog**, который будет представлять собой каталог компьютерных игр.

Программно создать несколько (10-20 различных) игр и добавить их в каталог.

В главном меню сделать выбор выпадающий список для игр каталога. При выборе игры становится активной кнопка “Показать данные”. В новом окне должно открыться изображение для обложки игры, данные о ней: название, жанр и т. д. Данные об играх читать из файлов в формате JSON (создать файлы при первом запуске).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “ComputerGames”.

Создать интерфейс **IGameCatalog**, который будет определять методы для работы с каталогом игр: для добавления и удаления игры.

Для класса **GameCatalog** модификатор partial. Создать ещё одну часть этого класса partial, реализующий интерфейс **IGameCatalog**.

Добавить в главное меню кнопку “Открыть каталог”. Открывается новое окно с таблицей игр. В таблице добавить возможность добавления и удаления игр. После закрытия окна обновлять игры в выпадающем списке.

Продвинутая часть (2 балла):

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать интерфейс **IStatistic** с методами, которые будут рассчитывать наивысшее, наименьшее, среднее и медианное качество продуктов в списке.

Создать 3-й partial класс **GameCatalog**. Он должен содержать методы для сортировки игр по названию и выбору игр по платформе и игровому режиму.

Добавить в главное меню два выпадающих списка. Первый позволяет выбрать платформу (PC, console, mobile), второй - режим игры (single, multi, online). В зависимости от выбранных значений изменять список игр. Список должен всегда сортироваться по названию в алфавитном порядке.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять набор фильтров и итоговый список игр. При смене формата копировать данные из "текущего формата" в файл "нового формата".

Проект “BoardGames”

Описание проекта

Реализовать приложение для поиска настольных игр. Проект должен содержать главное меню для выбора продуктов и окно с изображением (ссылки на источники можно указать в Readme) и описанием игры.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **BoardGame** со свойствами для названия, количества игроков, возрастного ограничения, описания и прочими необходимыми дополнительными свойствами.

Создать классы **CardGame**, **EuroGame**, **PartyGame**, (и прочие по желанию) которые будут наследоваться от **BoardGame**. В каждый добавить минимум по 1 уникальному полю.

Создать интерфейс **IGameCatalog**, который будет определять методы для работы с каталогом игр: для добавления и удаления игры.

Создать partial класс **GameCatalog**, который будет реализовывать интерфейс **IGameCatalog**.

Программно создать несколько (10-20 различных) игр и добавить их в каталог.

В главном меню сделать выбор выпадающий список для игр каталога. При выборе игры становится активной кнопка “Показать данные”. В новом окне должно открыться изображение для обложки игры, данные о ней: название, тип игры и т. д. Данные об играх читать из файлов в формате JSON (создать файлы при первом запуске).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “BoardGames”.

Создать интерфейс **IAnalyzer** с методами **AverageMinAmount()**, **AverageMaxAmount()**, **MeanAmount()**, **MeanAgeRestriction()**.

Создать 2-й partial класс **GameCatalog**. В нем реализуйте интерфейс **IAnalyzer**, чтобы определить среднее минимальное и максимальное количество игроков, средний диапазон мест и среднее возрастное ограничение по играм в каталоге.

Добавить в главное меню кнопку “Открыть каталог”. Открывается новое окно с таблицей игр. В таблице добавить возможность добавления и удаления игр. После закрытия окна обновлять игры в выпадающем списке.

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать интерфейс **IStatistic** с методами, которые будут рассчитывать наивысшее, наименьшее, среднее и медианное качество продуктов в списке.

Создать 3-й partial класс **GameCatalog**. В нем Создать методы для сортировки игр по названию и методы для фильтрации (выбору) игр по возрастному ограничению / по количеству игроков / виду (карточные, евро, пати и т. п.). Переопределите в классе **BoardGame** операторы сравнения для сортировки по названию.

Добавить в главном меню выпадающий список, который позволяет выбрать тип настольной игры. Добавить слайдер(ы) для настройки диапазона количества игроков: от 1 до 10. Добавить поле для указания минимального возраста. В зависимости от выбранных значений изменять список игр. Список должен всегда сортироваться по названию в алфавитном порядке.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять набор фильтров и итоговый список игр. При смене формата копировать данные из “текущего формата” в файл “нового формата”.

Проект “Checkers”

Описание проекта

Реализовать игру в шашки на 2-х игроков и против компьютера. Проект должен содержать стартовое меню и окно партии. При закрытии окна партии прогресс должен сохраняться на компьютере в файловую систему.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий положение и методы для движения (и, если нужно, атаки) фигуры.

Абстрактный класс фигуры, реализующий интерфейс и добавляющий необходимые свойства и методы. Если фигура после “съедания” фигуры соперника может съесть ещё одну, она **должна** это сделать (и так до тех пор, пока есть такая возможность).

Классы-наследники для каждого вида фигур: шашки и дамки.

Стартовое меню с 2-мя кнопками: “Новая игра на двоих” и “Продолжить игру”. Новая игра начинается белыми со стандартной расстановки фигур, а продолжение игры - с сохраненной партии, начиная с игрока, чей ход был до выхода.

В новом окне должно открываться игровое поле с фигурами. Управление мышью: выбор фигуры, перемещение фигуры. После этого происходит переход хода другому игроку.

Создать класс для сериализации информации в формате JSON. Сохранение игры осуществлять при закрытии окна игры.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей игры. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “Checkers”.

Для класса, отвечающего за логику партии добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации окончания игры при условии победы: один из игроков остался без фигур.

Сообщать пользователю о том, что одна из его шашек стала дамкой и может теперь перемещаться на любое количество клеток в любом направлении.

Добавить подсветку клеток, доступных для перемещения выбранной фигуры.

Добавить отмену хода: если поставить фигуру на то место, где она стояла, то можно походить другой фигурой.

В стартовом меню добавить поле для выбора папки для сохранения игры.

Продвинутая часть (2 балла):

Если выбранный файл сохранения не соответствует возможному варианту для игры (выбран какой-то сторонний файл), кнопка “Продолжить игру” должна быть неактивной. Также пользователю должно сообщать о том, что выбран файл некорректного формата.

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за фигуры и работу с ними, ход игры и т. п. поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить наследника для класса, отвечающего за логику игры. Заменить второго игрока компьютером. Он должен совершать случайное из всех возможных действий.

Добавить возможность "ничьи". При игре с человеком, если оба игрока нажмут на кнопку "Ничья" в свой ход, то игра завершается ничьей. При игре с компьютером, он соглашается, если имеет меньше очков, чем игрок. Шашка приносит 1 очко, дамка - 3.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить в главное меню 3-ю кнопку "Новая игра с компьютером".

Добавить в окне партии кнопки "Ничья" для обоих игроков (или одного в игре с компьютером).

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять прогресс активной партии. При смене формата копировать все данные из "текущего формата" в файл "нового формата".

Проект “PetShelter”

Описание проекта

Реализовать приложение для ведения учета животных в приюте. Проект должен содержать главное меню для выбора животных по фильтрам и окна с таблицей подходящих животных.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **Pet** со свойствами для клички, возраста, веса и других необходимых свойств.

Создать классы, наследующиеся от **Pet** наследников **Cat**, **Dog**, **Rabbit** (и прочие по желанию). Добавьте им по 2 новых поля.

Создать интерфейс **ICountable**, содержащий перегруженные методы **Count()**, **Count(Type)**, и метод **Percentage(Type)**, которые возвращают целые числа для информации о числе животных, числе животных конкретного типа и их % от общего числа.

Создать интерфейс **IFilter**, содержащий метод для фильтрации по виду животных **Filter(Type)**.

Создать класс **Shelter** со свойствами для названия, вместимости приюта, наличием открытой территории, реализующий интерфейсы **ICountable** и **IFilter**.

Программно создать несколько (10-20 различных) питомцев. Создать несколько приютов (3+) и поместить в них несколько питомцев (3+). Питомец не должен находиться в нескольких приютах одновременно.

В главном меню сделать выпадающие списки для выбора приюта и вида животного. Если не выбрано ни одно значение в списке, то учитывать все объекты данного типа. Создать кнопку “Показать питомцев”. В новом окне должна открываться таблица с питомцами, удовлетворяющие условиям фильтров. Данные о приютах читать из файлов в формате JSON (создать файлы при первом запуске). Данные таблицы сохранять в файле в стиле: “Подборка_№X_от_data” (или любом похожем).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “PetShelter”.

Для класса **Pet** модификатор **partial**. Создать ещё одну часть этого класса **partial** с конструктором, который в себя помимо прочего принимает параметр “клаустрофобия”. Аналогично сделать в классами-наследниками. У одного из классов сделать для данного поля значение по умолчанию **true**, у другого класса - **false**. Животных с “клаустрофобией” нельзя помещать в закрытый питомник.

Добавить для класса **Shelter** модификатор **partial**. Добавить перегрузку для метода фильтрации животных с клаустрофобией: **Filter(Type, bool)**.

Добавить в главное меню галочку для выбора приютов с открытым участком. Добавить в таблицу колонку для наличия или отсутствия “клаустрофобии” у питомца.

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать интерфейс **IChangeable** с методами для добавления и удаления животных в приют.

Создать ещё одну partial часть класса **Shelter**, реализующий интерфейс **IChangeable**.

Добавить в окне с таблицей питомцев возможность добавлять или удалять питомцев в случае, если выбран только один питомник. Также должен создаваться новый файл в формате json и xml, если нет животного с такими параметрами.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять отчет. При смене формата копировать данные из всех отчетов “текущего формата” в файлы “нового формата”.

Проект “TournamentTable”

Описание проекта

Реализовать приложение для ведения турнирной таблицы спортивных команд. Проект должен содержать главное меню для выбора вида спорта и сезона и окна с турнирной таблицей.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **Team** со свойствами для названия, побед, ничьих, поражений и других необходимых свойств. Определите свойство **CalculatePoints**, которое будет возвращать общее количество очков команды в турнире.

Создать классы **FootballTeam**, **BasketballTeam**, **VolleyballTeam**, наследующихся от **Team**. В каждом классе-наследнике переопределите свойство **CalculatePoints**, которое будет рассчитывать общее количество очков команды в соответствии с видом спорта.

Создать интерфейс **ITable**, который будет определять метод для добавления результатов матча между двумя командами **Match(Team, Team)**, и методы для сортировки по названию команды (в алфавитном порядке) и конечному количеству баллов (убыванию) **SortDefault()** и **SortByScore()**.

Создать класс **TournamentTable** со свойствами для названия, года сезона, массива команд, массива матчей и прочими необходимыми свойствами. Класс должен реализовать интерфейс **ITable**.

Программно создать несколько (10-20 различных) команд. Создать несколько турниров (3+) и поместить в них несколько команд (3+). Команды не должны находиться в турнирах разных видов спорта одновременно, но могут быть в турнирах разных сезонов одного вида спорта. Провести между всеми командами турнира матчи.

В главном меню сделать выпадающие списки для выбора вида спорта и номера сезона. Создать кнопку “Показать таблицу”, которая активируется, когда выбраны значения в обоих выпадающих списках. В новом окне должна открываться турнирная таблица, отсортированная в алфавитном порядке названий команд. Данные о турнирных таблицах читать из файлов в формате JSON (создать файлы при первом запуске). В окне рядом с таблицей расположить 2 кнопки: “Отсортировать по имени” и “Отсортировать по очкам”.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “TournamentTable”.

Добавить для класса **TournamentTable** модификатор partial. Создать ещё одну часть этого класса partial. В нём создать метод **Disqual(Team)**, который исключает команду из **дальнейшего** участия в турнире.

Доработать метод сортировки по очкам так, чтобы при равенстве очков критерий сортировки был - количество побед (или очков за победы), далее - количество ничей, далее - результат матча между двумя командами с одинаковыми очками. Если по всем

параметрам у команд совпадение, они занимают одно (более высокое место), а следующая после них команда имеет место на 2 ниже (т.е. пропустить место).

В окне с турнирной таблицей отображать места команд и учитывать подобные пропуски.

Продвинутая часть (2 балла):

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать ещё одну partial часть класса **TournamentTable**. В ней добавить метод **Getchanges(TournamentTable)**, который принимает в себя другой турнир и на выход выдает кортеж изменений мест команд, отсортированных в алфавитном порядке, включая: место в переданном турнире, место в текущем турнире, разницу в местах (например: +3 или -4). Если команда отсутствовала в прошлом турнире, то вместо места должен быть прочерк, а разница равна новому месту со знаком +. Если команда отсутствует в текущем турнире, то со знаком -.

Добавить в главное меню кнопку "Сравнительная таблица", которая становится активной, когда выбран вид спорта. При её нажатии открывается новое окно, в котором два выпадающих списка или чекбокса или слайдера для сезонов турниров в данном виде спорта. При выборе обоих сезонов должна активироваться кнопка "Показать таблицу". При нажатии на неё должна отображаться сравнительная таблица изменений мест команд из второго сезона относительно первого.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять файлы с информацией о турнире. При смене формата копировать данные из всех отчетов "текущего формата" в файлы "нового формата".

Проект “InteriorCatalog”

Описание проекта

Реализовать приложение для создания каталога мебели. Проект должен содержать главное меню для выбора каталога, окна с каталогом и окна с предметом мебели (ссылки на источники можно указать в Readme).

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **Furniture** со свойствами для *уникального кода*, артикула товара, бренда/производителя, модели и других необходимых свойств. Определите свойство **Price**, которое будет возвращать цену предмета мебели.

Создать классы **Chair**, **Table**, **Sofa**, **Bed**, наследующихся от **Furniture**. В каждом классе-наследнике добавьте по 2 независимых поля, которые будут влиять на цену товара. В каждом классе-наследнике переопределите свойство **Price**, которое будет рассчитывать цену с учетом базовой цены и новых полей классов-наследников.

Создать интерфейс **IFurnitureCatalog**, который будет определять методы для работы с каталогом мебели: добавление одного или нескольких предметов и удаление конкретного предмета (по артикулу) или всех предметов определенного вида, **Add(Furniture)**, **Add(FurnitureItem[])**, **Remove(string)**, **Remove(Type)**.

Создать класс **FurnitureCatalog** со свойствами для названия, сезона каталога, массива мебели и прочими необходимыми свойствами. Класс должен реализовать интерфейс **IFurnitureCatalog**.

Программно создать несколько (10-20 различных) предметов мебели. Создать несколько каталогов (3+) и поместить в них несколько предметов (3+). Одни и те же предметы мебели могут находиться в различных каталогах.

В главном меню сделать выпадающие списки для выбора каталога из списка. Создать кнопку “Показать каталог”, которая активируется, когда выбран какой-либо каталог. В новом окне должна открываться таблица с предметами мебели из данного каталога, отсортированная по возрастанию артикула. Данные о предметах мебели читать из файлов в формате JSON (создать файлы при первом запуске).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “InteriorCatalog”.

Создать интерфейс **ISortable** с методами сортировки **Sort(bool)**, **SortByName(bool)** и **SortByPrice(bool)** по возрастанию или убыванию (по умолчанию) артикула, названия и цены товара соответственно.

Добавить для класса **FurnitureCatalog** модификатор partial. Создать ещё одну часть этого класса partial. В нем реализовать интерфейс **ISortable**.

В окне с каталогом добавить кнопки для сортировок таблицы с каталогом по возрастанию и убыванию артикула, названия и цены товара соответственно.

Добавить в таблицу колонку с кнопкой или же отдельную кнопку, привязываемую к выделенной строке таблицы, при нажатие на которое открывается изображение предмета мебели в новом окне.

Продвинутая часть (2 балла):

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать ещё одну partial часть класса **FurnitureCatalog**. В нем создать метод **PrioritySort()**, который сортирует каталог сперва по бренду, потом по модели, потом по названию, потом по артикулу в алфавитном порядке, так что объекты становятся "сгруппированы".

Создать от **Chair** классы-наследники **Armchair** и **Stool**. Добавить им по 1+ свойству и переопределить с его учетом свойство **Price**.

Добавить в окне с каталогом выпадающий список для выбора списка мебели. Если в нем выбрано какое-то значение, то в таблице выводить только товары данного вида.

Добавить кнопку "Сгруппировать", при нажатии на которую, таблица сортируется с помощью метода **PrioritySort()**.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять файлы с информацией о предметах мебели. При смене формата копировать данные из всех отчетов "текущего формата" в файлы "нового формата".

Проект “SalesReport”

Описание проекта

Реализовать приложение для создания отчета отдела продаж электронной техники. Проект должен содержать главное меню для выбора периода и вида техники и окна с отчетом по продажам.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать класс **ITProduct** со свойствами для **уникального кода**, артикула товара, бренда/производителя, модели, даты продажи (null, если не продан ещё) и других необходимых свойств. Определите свойство **Price**, которое будет возвращать цену устройства.

Создать классы **Laptop**, **Smartphone**, **Tablet**, наследующихся от **ITProduct**. В каждом классе-наследнике добавьте по 2 независимых поля, которые будут влиять на цену устройства. В каждом классе-наследнике переопределите свойство **Price**, которое будет рассчитывать цену с учетом базовой цены и новых полей классов-наследников.

Создать интерфейс **IReportable**, который будет определять методы для работы с отчетом о продажах: сортировка по артикулу по возрастанию или убыванию **Sort(bool)**, выбор продукции только определенного типа (или всех устройств, если передан базовый тип) **Select(Type)**.

Создать класс **Report** со свойствами для названия, периода, массива устройств и прочими необходимыми свойствами. Класс должен реализовать интерфейс **IReportable**.

Программно создать несколько (10-20 различных) устройств. Создать несколько отчетов (5+) и поместить в них несколько устройств (2+). Одни и те же устройства не могут находиться в различных отчетах (продать можно 1 раз).

В главном меню сделать выпадающий список для выбора вида устройств и период отчета (день/неделю/месяц/квартал и т.п.). Выбрать с помощью календаря дату. Создать кнопку “Показать отчет”, которая активируется, когда выбран период отчета и дата. В новом окне должна открываться таблица с устройствами, проданными за данный отчетный период, отсортированная по возрастанию артикула. В окне должны быть кнопки для изменения направления сортировки: по убыванию и возрастанию. Данные об устройствах читать из файлов в формате JSON (создать файлы при первом запуске). Данные таблицы сохранять в файле в стиле: “Отчет_№X_за_data2-data2” (или любом похожем).

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей программы. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “SalesReport”.

Добавить для класса **Report** модификатор partial. Создать ещё одну часть этого класса partial. В нем создать ещё один конструктор, который получает на вход массив отчетов и даты начала и конца, за которые нужно собрать отчет и собирает все проданные товары в указанном временном промежутке (исключая дублирование). Создать в этом же классе методы для добавления одного или нескольких устройств в массив проданных. Добавить

метод, принимающий в себя другой отчет и добавляющий проданные товары из него (исключая дублирование).

В главном меню добавить чекбокс отчетов, который формируется на основе выбранной даты и периода, в котором есть хотя бы 1 товар, проданный в указанный период.

Теперь кнопка “Показать отчет” должна становиться активной, если выбран хотя бы один отчет из чекбокса. В таблицу должны загружаться все товары из выбранных отчетов (исключая дублирование).

Продвинутая часть (2 балла):

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за основные классы и работу с ними, поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации информации в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Создать ещё одну partial часть класса **Report**. В нем создать метод **GetPriceChanges(string)**, который возвращает массив изменений цены на какой-либо товар определенного товара (с указанием даты продажи и средней цены продажи в эту дату), отсортированный в хронологическом порядке.

Добавить в окне с каталогом выпадающий список для выбора артикула устройства. Если в нем выбрано какое-то значение, то активировать кнопку “Динамика цены”. При нажатии на кнопку должен открываться график с датами по оси абсцисс и ценой по оси ординат.

Добавить в главном меню выпадающий список для выбора формата, в котором сохранять файлы с информацией об устройствах. При смене формата копировать данные из всех отчетов “текущего формата” в файлы “нового формата”.

Проект “Sapper”

Описание проекта

Реализовать игру в сапера. Проект должен содержать стартовое меню и окно игры. При закрытии окна игры прогресс должен сохраняться на компьютере в файловую систему.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейс, описывающий свойства клетки.

Абстрактный класс клетки, реализующий интерфейс и добавляющий необходимые свойства и методы.

Классы-наследники: мина, флаг, ячейка с числом, пустая ячейка (вокруг которой нет мин).

Класс поля со свойствами для размера: количества ячеек по горизонтали и вертикали.

Количество мин должно быть в диапазоне от 20% до 40% всех ячеек.

Если игрок попал на мину, то игра заканчивается проигрышем.

Стартовое меню с 2-мя кнопками: “Новая игра” и “Продолжить игру”. Новая игра начинается с пустого поля 5x5, а продолжение игры - с сохраненного поля.

В новом окне должно открываться игровое поле с размеченными клетками. Управление мышью: левая кнопка - активировать поле, правая - поставить флаг.

Создать класс для сериализации информации в формате JSON. Сохранение игры осуществлять при закрытии окна игры.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей игры. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “Sapper”.

Для класса, отвечающего за логику игры добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации победы, если все мины закрыты флагами, а остальные ячейки поля активированы.

При генерации клеток учесть, чтобы каждую мину окружало не менее 3 клеток с числами (иначе углы или другие части могут быть “заблокированы”).

В главном меню добавить поле для выбора папки для сохранения игры.

В главном меню добавить возможность выбора размера поля (5x5, 10x10, 5x10 и т.п.). Масштабировать размер ячеек и поля при создании новой игры в зависимости от установленного размера. Можно сделать списком, слайдером или текстовыми полями с ограничениями.

Продвинутая часть (2 балла):

Если выбранный файл сохранения не соответствует возможному варианту для игры (выбран какой-то сторонний файл), кнопка “Продолжить игру” должна быть неактивной. Также пользователю должно сообщать о том, что выбран файл некорректного формата.

Создать в библиотеке "Model" папки "Core" и "Data". Файлы, отвечающие за игровые элементы и работу с ними, ход игры и т. п. поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Создать ещё одну часть этого класса partial для реализации таймера, который отсчитывает время, оставшееся у игрока для прохождения игры. Если таймер достигает значения 0, заканчивать игру поражением.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять прогресс активной партии. При смене формата копировать все данные из "текущего формата" в файл "нового формата".

Добавить выпадающий список для выбора "уровня сложности", который определяет, как много мин будет в процентном отношении относительно общего количества клеток.

Добавить таблицу результатов лучших 10 игр. Результат определяется по формуле, включающей размер поля, уровень сложности и время, которое игрок потратил на прохождение уровня.

Проект “PingPong”

Описание проекта

Реализовать игру в пинг-понг на 1-го и/или 2-х игроков. Проект должен содержать стартовое меню и окно игры. При закрытии окна игры прогресс должен сохраняться на компьютере в файловую систему.

При выполнении работы проверяйте, соблюдаете ли вы пункты в критериях оценивания задания на Moodle.

Базовая часть (5 баллов):

Создать интерфейсы, описывающие игровые элементы: ракетка, мяч.

Абстрактный класс ракетки, реализующий интерфейс и добавляющий необходимые свойства и методы.

Классы-наследники: деревянная, кожаная, полиуретановая. У них разная сила подачи, удара, закрутки мяча.

Аналогично сделать разные классы для мячей.

Класс поля со свойствами для размера: количества ячеек по горизонтали и вертикали.

Игрок с помощью элементов управления контролирует положение ракетки. Для старта игры требуется нажатие клавиши для подачи, далее шар отскакивает от ракетки автоматически, если попадает в область ракетки. Проигрыш – вылет со стола или попадание в сетку.

Стартовое меню с 2-мя кнопками: “Новая игра” и “Продолжить игру”. Новая игра начинается с пустого поля и подачи, а продолжение игры - с сохраненного поля и подачи. При выходе в главное меню сохраняется текущий счет. Если выход осуществлен во время розыгрыша, этот розыгрыш не учитывается.

Добавить возможность настройки или отображения управления.

Создать класс для сериализации информации в формате JSON. Сохранение игры осуществлять при закрытии окна игры.

Дополнительная часть (3 балла):

Создать и подключить к проекту библиотеку “Model”. В неё поместить все файлы, отвечающие за бизнес-логику вашей игры. Классы, отвечающие за визуальную составляющую и взаимодействие с игроками оставить в основном проекте “PingPong”.

Для класса, отвечающего за логику игры добавить модификатор partial. Создать ещё одну часть этого класса partial для реализации победы, если счет достигает указанного в меню значения (11 или 21 очков) и имеет отрыв не менее, чем в 2 очка от противника.

В главном меню добавить поле для выбора папки для сохранения игры.

В главном меню добавить возможность выбора продолжительности матча (11 или 21 очков). Также продолжительность влияет на количество подач игрока: 2 и 5 соответственно. Можно сделать списком или текстовым полем с ограничением вводимых данных.

Продвинутая часть (2 балла):

Если выбранный файл сохранения не соответствует возможному варианту для игры (выбран какой-то сторонний файл), кнопка “Продолжить игру” должна быть неактивной. Также пользователю должно сообщать о том, что выбран файл некорректного формата.

Создать в библиотеке “Model” папки “Core” и “Data”. Файлы, отвечающие за элементы игры и работу с ними, ход игры и т. п. поместить в первую папку, а файлы, отвечающие за сериализацию и десериализацию - во вторую.

Добавить класс для сериализации состояния игры в формате XML. Сделать общий абстрактный класс для сериализации, от которого должны наследоваться классы для сериализации в JSON и XML.

Добавить в главное меню выпадающий список для выбора формата, в котором сохранять прогресс активной партии. При смене формата копировать все данные из “текущего формата” в файл “нового формата”.

Создать ещё одну часть этого класса partial для реализации игры вдвоем. Добавить в главное меню возможность выбора одиночной игры или вдвоем. Предоставление настройки или показа альтернативных кнопок управления второму игроку.